

Trajectory Classification with LSTM for Robotic Peg-in-Hole

Group 1 Project

UE5 Apprentissage

Master's Machine Learning
Project – 2025–2026





Project Context

- ▶ Course project: classify a full robot trajectory into **3 classes** with a sequence model.
- ▶ Data: peg-in-hole task from a 7-DoF robotic arm, with temporal observations over 150 steps.
- ▶ Core objective from assignment:
 - ▶ achieve strong test performance,
 - ▶ analyze the effect of modeling and data choices.
- ▶ Course link: the approach follows UE5 sequence modeling (RNN/LSTM module).



Roadmap (10 minutes)

1. Data extraction and preparation
2. Model
3. Experimental plan for hyperparameters
4. Challenges encountered
5. Results and analysis
6. Conclusion

Suggested pacing: ~ 3 min data+model, ~ 2 min plan+issues, ~ 3.5 min results, ~ 1.5 min wrap-up.

All figures are taken from project outputs in `images/`.



Dataset Structure

- ▶ Files: `class1_trajectories.npy`, `class2_trajectories.npy`, `class3_trajectories.npy`
- ▶ Balanced dataset: **256 trajectories per class**, total **768 trajectories**
- ▶ Tensor shape per class: (256, 150, 20)
- ▶ Label encoding: class 1→0, class 2→1, class 3→2

Observation block	Content	Size
peg_pose	position + quaternion	7
peg_vel	linear + angular velocity	6
hole_pos	position + quaternion	7
Total features		20



Preprocessing Pipeline

- ▶ Reproducible setup: fixed seed (SEED=42) for split and initialization.
- ▶ Split strategy: **80/10/10** $\Rightarrow$ train 614, val 76, test 78.
- ▶ Train loader is shuffled; validation/test loaders are deterministic.
- ▶ Z-score normalization computed on train set and reused on val/test:

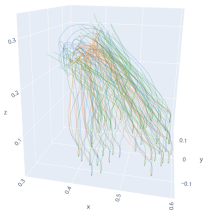
$$\mathbf{x}_{norm} = \frac{\mathbf{x} - \mu_{train}}{\sigma_{train} + \epsilon}$$

- ▶ Numerical stability: replace zero std by small ϵ .



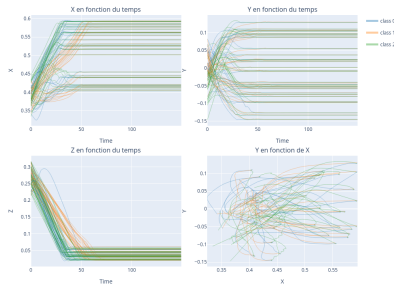
Trajectory Exploration Before Training

Trajectoires (position du peg) par classe



3D trajectory view (x, y, z) by class.

Trajectoires par composante



Temporal/component projections
 $(x(t), y(t), z(t), y(x))$.

Initial separability patterns support using a temporal classifier rather than a static model.



Why LSTM for This Task?

- ▶ The class depends on **trajectory dynamics**, not one isolated timestep.
- ▶ From UE5 course (3_RNN_Transformers_2025.pdf): vanilla RNN training suffers from vanishing/exploding gradients.
- ▶ LSTM introduces a memory cell with gates to control information flow:

$$i_t, f_t, o_t = \sigma(\cdot), \quad \tilde{c}_t = \tanh(\cdot), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

- ▶ This is aligned with course motivation: preserve useful long-term information.



Final Architecture and Training Setup

- ▶ Input: sequence (batch, 150, 20)
- ▶ Backbone: LSTM (num_layers=2, hidden_size=64, dropout=0.2)
- ▶ Sequence-to-class head: last hidden state $\rightarrow$ Linear(64, 3)
- ▶ Loss: CrossEntropyLoss
- ▶ Optimizer: Adam(lr=1e-3)
- ▶ Batch size: 32, training budget: up to 40 epochs

Experimental Plan for Hyperparameters



Planned Ablations

Run	Main setting	Purpose
R1	hidden=100, z-score, 10 epochs	baseline behavior and convergence
R2	hidden=64, no normalization	test sensitivity to scaling choices
R3	hidden=64 + dropout	measure regularization effect
R4	R3 + gradient clipping	stabilize training dynamics

Additional fixed controls: seed, train/val/test split, optimizer family, and class definition.



Evaluation Protocol

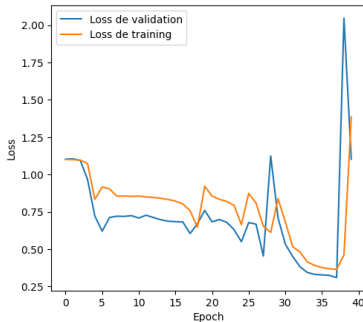
- ▶ Primary metric: **test accuracy** on held-out internal test set (78 trajectories).
- ▶ Supporting metrics: train/val loss and train/val accuracy across epochs.
- ▶ Selection logic: keep the simplest setting that maximizes generalization and stability.
- ▶ Assignment requirement considered: model interface kept compatible with teacher-provided unseen test tensor format.

Interpretation caution: final numbers are taken from documented run logs (`rapport.ipynb/rapport_final.ipynb`); external teacher test remains the final robustness check.

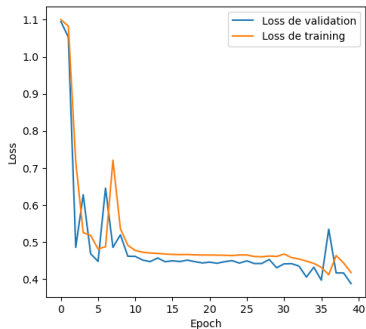


Practical Issues During Development

- **Reproducibility:** without fixed seeds, run-to-run comparison was unreliable.
- **Optimization sensitivity:** preprocessing and regularization strongly changed convergence behavior.



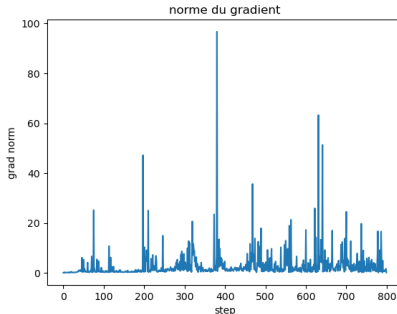
Without normalization: noisier convergence.



Without dropout: weaker regularization behavior.



Exploding Gradients and Fix



Observed spikes in gradient norm across training steps.

Problem

Large gradient spikes destabilized updates.

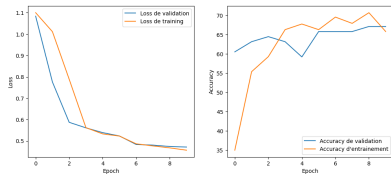
Mitigation

Gradient clipping in the training loop with max norm 1.0:

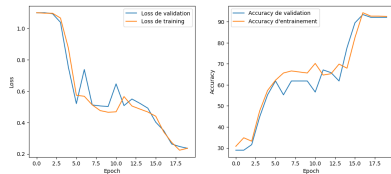
PyTorch `clip_grad_norm(...)`.



Runs R1 and R2



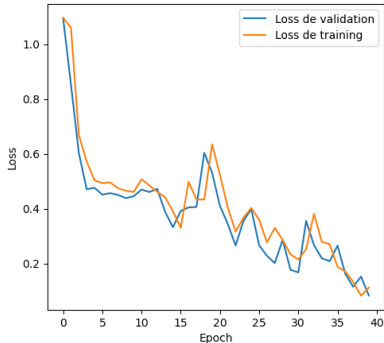
R1: baseline, limited performance and unstable validation behavior.



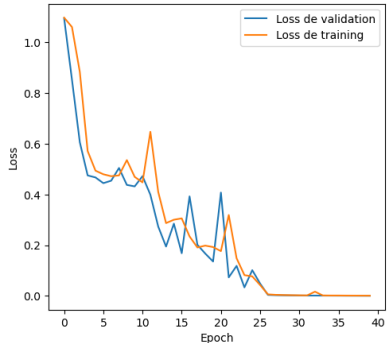
R2: major improvement, **92.31%** test accuracy (72/78).



Runs R3 and R4



R3 (dropout): run logs report **98.72%** test accuracy (77/78).



R4 (dropout + clipping): run logs report the best result, **100%** (78/78).



Quantitative Summary and Interpretation

Run	Test accuracy	Key effect
R1	low baseline	underfit/unstable baseline
R2	92.31% (72/78)	better capacity choice
R3	98.72% (77/78)	dropout improves generalization
R4	100.00% (78/78)	clipping improves stability

- ▶ Most impactful additions: **dropout** + **gradient clipping**.
- ▶ `classification.ipynb` currently stores another unstable snapshot ($\sim 32\%$ test), so final comparisons rely on report logs.
- ▶ Internal split results are excellent; external hidden test set is still required for final robustness validation.



Conclusion

- ▶ We built an LSTM-based trajectory classifier aligned with UE5 sequence modeling principles.
- ▶ In the documented experiment logs, the final recipe (2-layer LSTM, dropout, clipping) reached perfect internal test performance.
- ▶ The experiment plan made performance gains explainable, not only empirical.
- ▶ Main engineering lesson: stability controls are critical for recurrent models.



Next Steps

- ▶ Run official teacher-held test evaluation to confirm out-of-sample robustness.
- ▶ Complete the assignment analysis on **feature influence** (e.g., pose-only, velocity-only, reduced subsets).
- ▶ Report class-wise metrics and confusion matrix on external test for deeper error analysis.

Material sources: `classification.ipynb`, `rapport_final.ipynb`,
`consignes groupe 1/README_classif_lstm_students.md`,
`3_RNN_Transformers_2025.pdf`.